Tyler - Green

Ben - Red

Samuel - Blue

Taylor – Orange

Full Stack Development — Team Lab

React Innovation Challenge: Build Your Own Interactive Application

Team Lab

React + Vite

Pod Teams

Lab Overview:

Your development team has been hired to create a new interactive web application. The client has given you considerable creative freedom, but the application must demonstrate modern front-end development techniques.

Unlike Campus Pulse, where everyone developed the same application, your team will choose its own application theme, design, features, and data.

The challenge is to build something useful, original, and interactive while demonstrating the React and JavaScript concepts learned to this point in the semester.

Important: This is a front-end application. You are not required to develop a backend, connect to a database, or implement user authentication.

Part 1 — Choose Your Application

Each team must select a unique application concept. Your application should solve a problem, organize information, simulate a real-world activity, or help users make decisions.

Theme Proposal

Before programming, each team must submit a brief proposal explaining:

1. What is the application's name?
    1. Impact Hub
2. What problem does it solve?

    1. People in need who need volunteer help can post their requests on the hub for people to accept and help them with what they need.

3. Who would use it?
    1. This program is designed for members of the community to post volunteer requests or events to obtain need. Anyone viewing the page can then elect to volunteer for a particular event they would like to help with.
4. What type of information will it manage?
    1. Requests
        1. Name of request
        2. Person/Organization making the request
        3. Status (priority level)
        4. Date and time of request
        5. Location
        6. Category
        7. Description of request
        8. Number of Volunteers
    2. Total Volunteers
    3. Total Requests in Open status
    4. Total Requests in Urgent status
5. What can users create, view, update, and delete?

    Create – events that someone needs help with

    View – cards representing each volunteer request's information that's posted

    Update – update any of an event's existing information except status

    Delete – completely remove an entire event and its information from the list

    Volunteer – click a button to volunteer for an event
6. What part will each team member play in this development lifecycle?

    Each team member's play within the development cycle will be documented throughout this document with text colors. Later sections involving code will be

determined as we develop the page. Each member will also be assigned components in which they are responsible.

Part 2 — Technical Requirements

Your application must demonstrate the concepts covered in our HTML, CSS, JavaScript, and React lectures.

Concept	Minimum requirement
✓ HTML/CSS	Well-structured, styled, responsive interface
✓ JavaScript	Arrays, objects, functions, and array methods
✓ React components	At least 5 reusable components
✓ Props	Pass information between components
✓ useState	Manage at least 3 meaningful state values
✓ Event handling	Buttons, inputs, and other user interactions
✓ Conditional rendering	Display different content based on application state
✓ Lists and keys	Dynamically render collections of data
✓ Forms	Controlled inputs with validation
✓ CRUD	Create, Read, Update, and Delete records
✓ Search	Search records by text
✓ Filtering	Filter records by category or status
✓ useEffect	Load initial data and demonstrate an effect
✓ JSON	Store at least 10 realistic starter records
✓ Immutable updates	Use map(), filter(), and spread syntax appropriately

*** All Requirements Accounted For

Your team should not simply rename Campus Pulse components. Their component structure should reflect the purpose of their own application.

Required CRUD Functionality

✔ Create

✔ Add a record

✔ Read

✔ Update

✔ Delete - Remove a record

- All functionalities accounted for

Every CRUD operation must visibly update the React interface without reloading the page.

For this lab, it is acceptable for changes to exist only in memory. Refreshing the browser may reset the application to its initial JSON data.

Part 3 — Development Process

Teams must follow our established software development process rather than immediately beginning to code. Documentation is important at this point and should be complete and professional.

Step 1 — Document and Divide & Conquer

Identify the application's main features, divide the work into smaller tasks, sketch the user interface, and determine which React components are needed.

Create a component hierarchy showing parent-child relationships and where shared state will live.

Main Features:

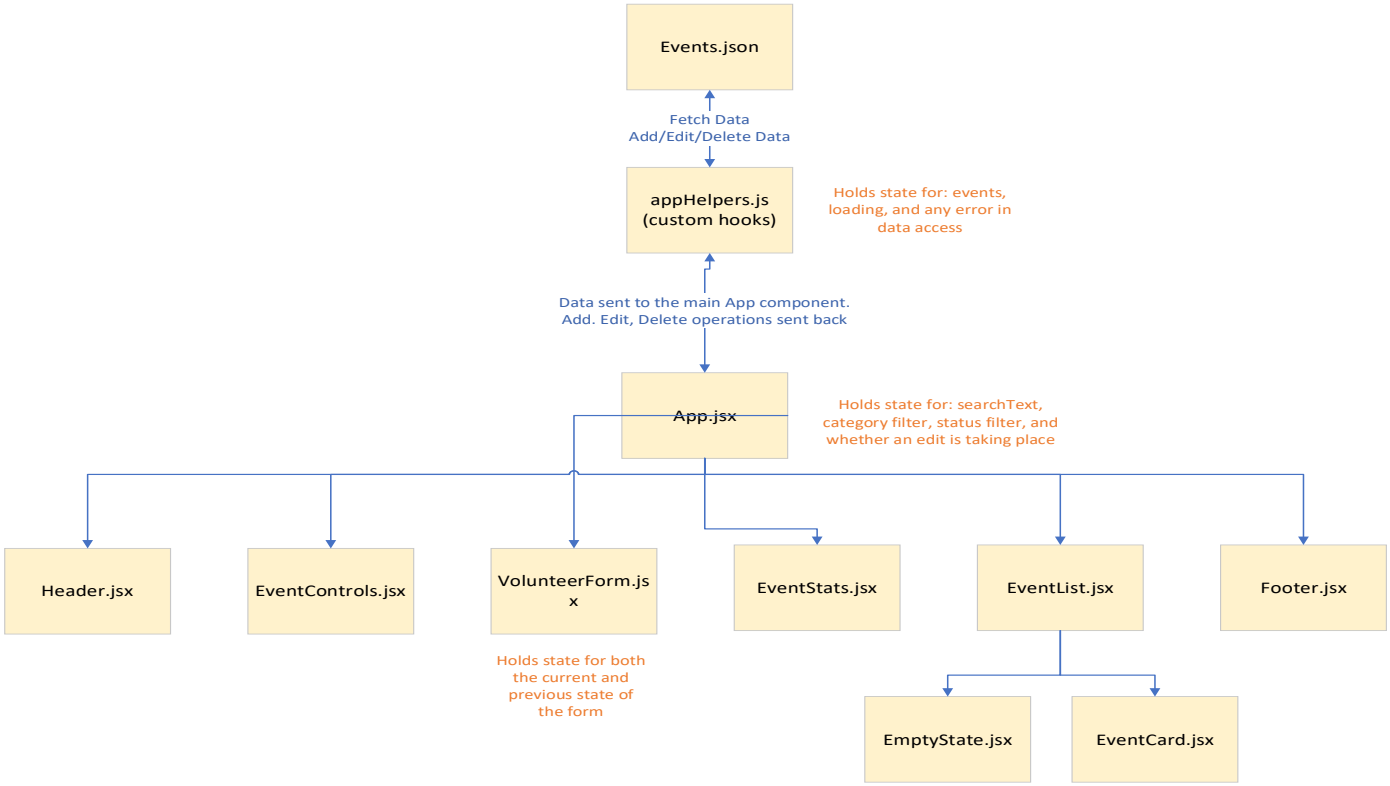
1. A form for adding/editing a volunteer posting
2. Main display to view all open volunteer postings
3. Search postings by applicable criteria

4. Filtering <select> drop-down list
5. Delete button for removing volunteer postings
6. Button to volunteer for an event

Components:

1. VolunteerForm - Ben
    a. Handles BOTH creating and editing an event. When editing, it receives the selected event and prepopulates the fields. When creating, it starts with blank fields.
2. EventControls
    a. Contains the search input, filter dropdown, and add-event button. It receives the current search and filter values and callbacks from the main App component
3. Pagenation
    a. Displays and handles the controls for navigating between each page of event cards
4. EventStats
    a. Shows summary statistics, such as the number of open requests, total volunteer sign-ups, or urgent events. These values can be calculated from the event collection rather than maintained as separate state.
5. EventList
    a. Receives the events that should be displayed and renders them using .map(). It can also display the EmptyState component when filtering the full list produces no results
6. EventCard
    a. Displays an individual event's information
7. Header?
    a. Displays the Impact Hub Logo and title, a short description, and a call to action
8. Footer?
    a. Displays a brief mission statement, credits, or team information
9. EmptyState
    a. Provides a friendly message when no events exist, or a search returns no matches. It can optionally offer a button to clear the search and filters.

Component Hierarchy with State Management:



Step 2 — Design the Algorithms

Write pseudocode for adding, editing, deleting, searching, and filtering records.

Explain how the application will update arrays without directly mutating React state.

Pseudocode

Function	Pseudocode
addEvent	1. Define the function inside useCallback so reference to it is cached until its dependencies change 2. Define the function to accept a new volunteer event to be added 3. convert the event into an object by spreding all its key/values out 4. Get the latest list of events 5. Add the new event to the full list of events by spreading out the current list and adding the new one to end of the list.
editEvent	1. Define the function inside useCallback so reference to it is cached until its dependencies change 2. Call the setEvents state callback for setting up the events array 3. Get the latest list of volunteer events

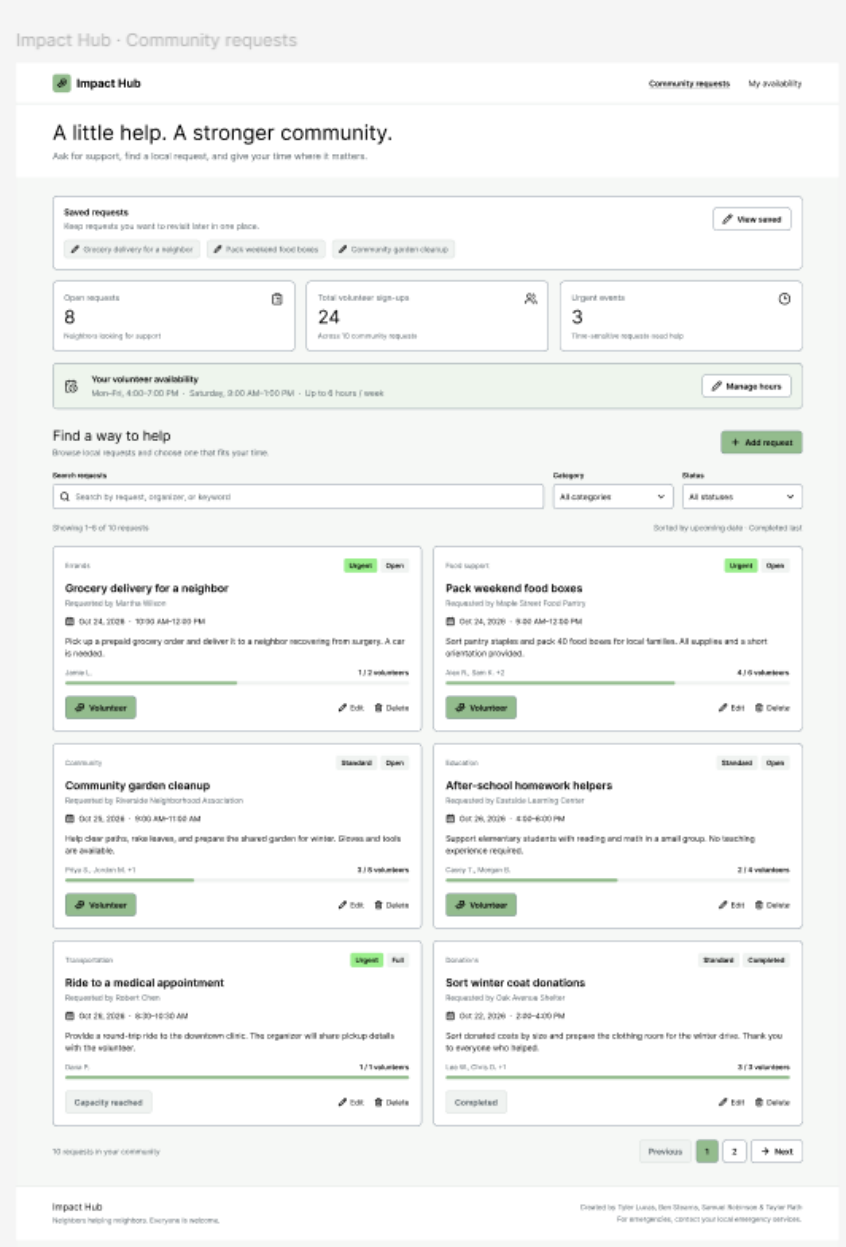
	4. use .map() to set the current list to a new array 5. Check every event in the current list 6. If the event's id isn't equal to the one being updated, keep it as is 7. If the event's id matches the one being updated, merge the old event's data with the new data
deleteEvent	1. Define the function inside useCallback so reference to it is cached until its dependencies change 2. Get the current list of volunteer events 3. Filter that list to only keep ones that don't match the id of the one being deleted
searchEvents	1. Define as useMemo so that it only recalculates the events to display if events, search text, category filter, or status filters are changed by the user 2. Go through each event from the full list and start filtering 3. Clean up the user's entered search criteria by removing leading/trailing whitespace and cast to lowercase 4. If the user didn't enter any search criteria, that event automatically passes 5. If the user did type in something, check if any of its relevant information was a match for the user's entered criteria 6. If the event was a match for the user's criteria as well as their category and status filters, keep the event. Otherwise don't keep it.
filterEvents	1. This is already been baked into the searchEvents function. 2. At the very end of the checking whether an event is a match for what the user typed into the search box, only return that matching event if it's also a match for the user's selected category or status filters as well.

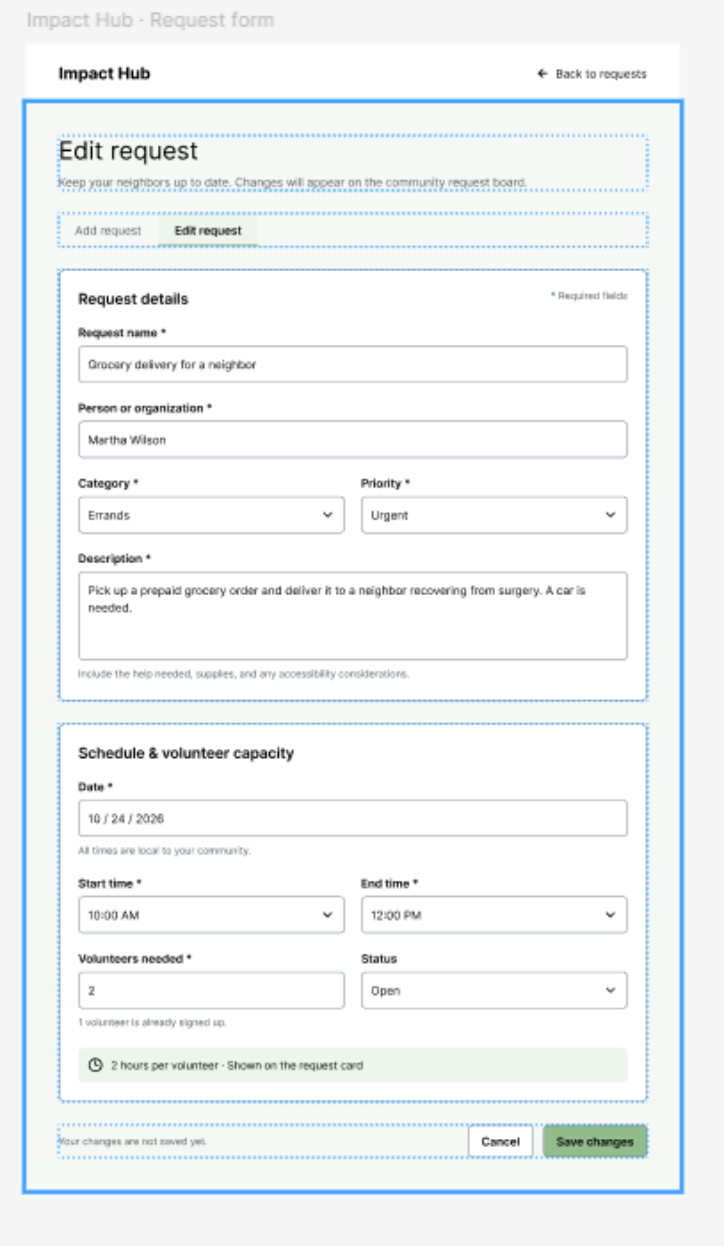
Step 3 — Build the Application

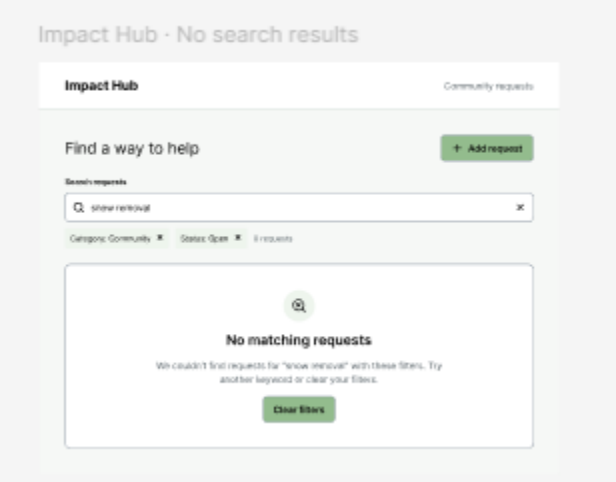
Primary Color Scheme:

DarkSeaGreen	LawnGreen	SlateGrey	Black
Primary	Accent/Secondary	Borders	Text
	LightGreen		
	Alternate Accent/Secondary		

Wireframe:




Impact Hub · Request form
Impact Hub
Back to requests
Edit request
Keep your neighbors up to date. Changes will appear on the community request board.
Add request
Edit request
Request details
* Required fields
Request name *
Grocery delivery for a neighbor
Person or organization *
Martha Wilson
Category *
Errands
Priority *
Urgent
Description *
Pick up a prepaid grocery order and deliver it to a neighbor recovering from surgery. A car is needed.
Include the help needed, supplies, and any accessibility considerations.
Schedule & volunteer capacity
Date *
10 / 24 / 2026
All times are local to your community.
Start time *
10:00 AM
End time *
12:00 PM
Volunteers needed *
2
Status
Open
1 volunteer is already signed up.
2 hours per volunteer · Shown on the request card
Your changes are not saved yet.
Cancel
Save changes



Click Here to View Wireframe Online

Create a Vite project, develop the components, load starter JSON data, implement state management, and connect the components through props and callback functions.

Create your new GitHub repository so everyone can contribute their code.

Build one working feature at a time.

Project Structure:

impact-hub/

├── public/

│ └── events.json - Ben

├── src/

│ ├── components/

│ │ ├── Header.jsx - Samuel

│ │ ├── EventStats.jsx - Taylor

│ │ ├── EventControls.jsx - Ben

│ │ ├── EventList.jsx - Tyler

│ │ ├── EventCard.jsx - Tyler

│ │ ├── VolunteerForm.jsx - Ben

│ │ ├── EmptyState.jsx - Ben

│ │ ├── Pagination.jsx - Ben

│ │ └── Footer.jsx - Samuel

│ ├── data/

│ │ └── eventHelpers.js – Ben and anyone else that needs it

│ ├── App.jsx - Taylor

│ ├── App.css - Taylor

│ ├── index.css – Samuel and anyone else who needs to tweak it

│ └── main.jsx – No edits needed

├── index.html– No edits needed

├── package.json– No edits needed unless someone wants to install other packages

├── vite.config.js– No edits needed

└── README.md - Ben

Step 4 — Test and Improve

Test each CRUD operation, empty inputs, search results, filters, and the behavior when no records match.

Fix errors and verify that the browser console contains no unresolved application errors.

Part 4 — Team Responsibilities

Each team member must make meaningful contributions to both development and testing.

Every team member must be able to explain at least one component, one state variable, and one event-handling function during the demonstration.

Each team must implement at least one additional feature that makes its application more interesting.

Examples include a favorites system, sorting controls, a progress indicator, summary statistics, a random recommendation, or a custom status workflow.

The feature must involve React state or meaningful JavaScript logic, not simply additional styling.

Job Distribution

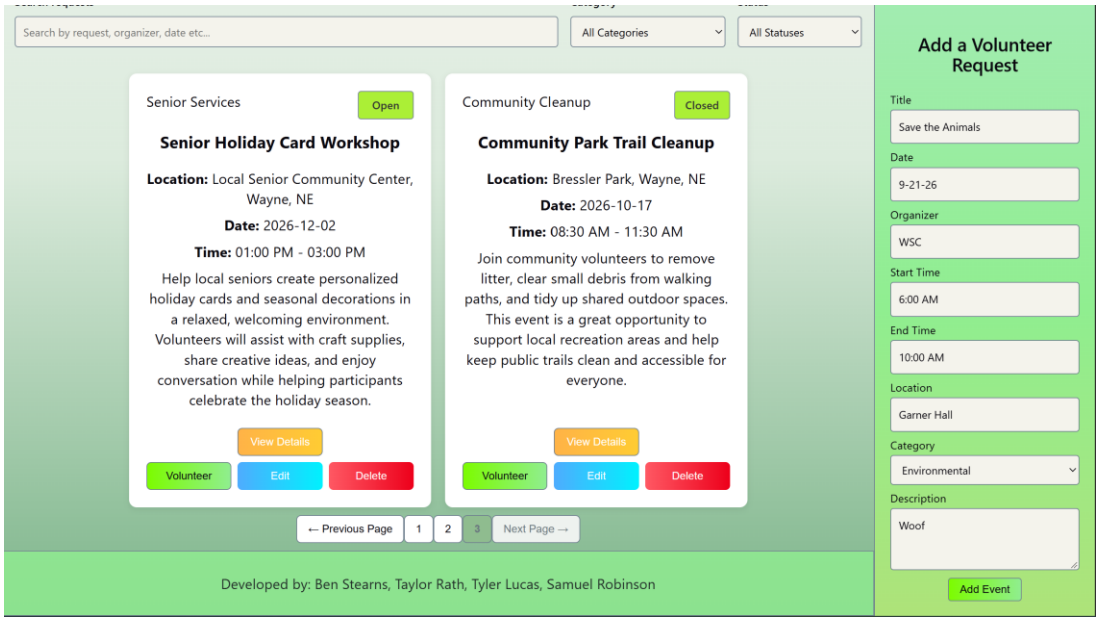
Team Member	Jobs
Ben Stearns	• VolunteerForm.jsx • EventControls.jsx • EmptyState.jsx • Pagination.jsx • Events.json • Readme • Testing • Associated css
Taylor Rath	• EventStats • App jsx • Associated css
Tyler Lucas	• EventList • EventCard
Samuel Robinson	• Header.jsx • Footer.jsx • Index.css • Logo image

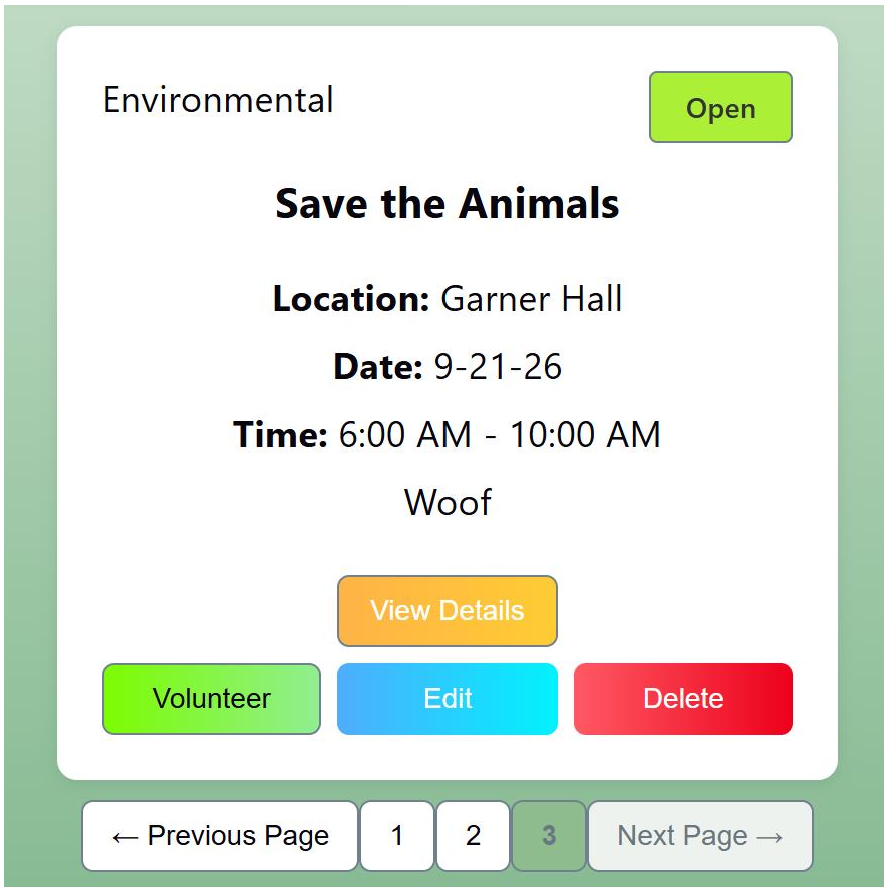
Part 5 — Testing Documentation

Teams must submit evidence of testing. (This will be part of your class demonstration)

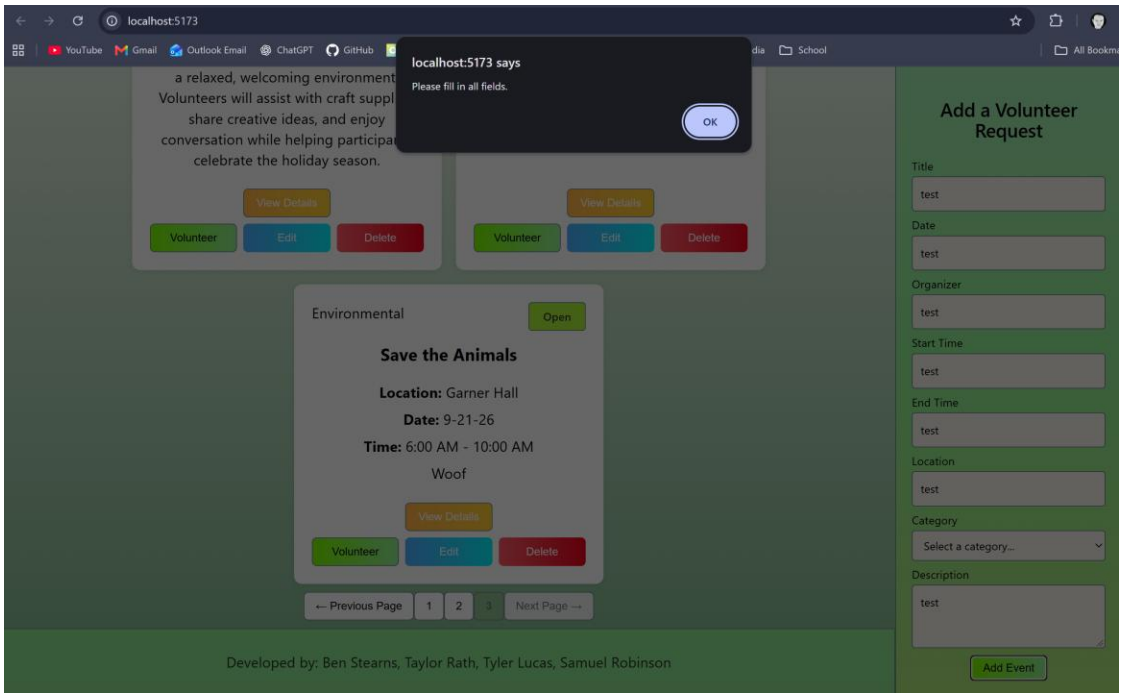
Test	Expected result
Add a valid record	New record appears
Submit an empty required field	Validation prevents submission
Edit an existing record	Updated information appears
Delete a record	Record disappears
Search for a known record	Matching records display
Search for nonexistent text	Empty-state message appears
Select a filter	Only appropriate records display
Reset search and filter	All current records display

Students must record actual results and document at least two problems they encountered and how they resolved them.

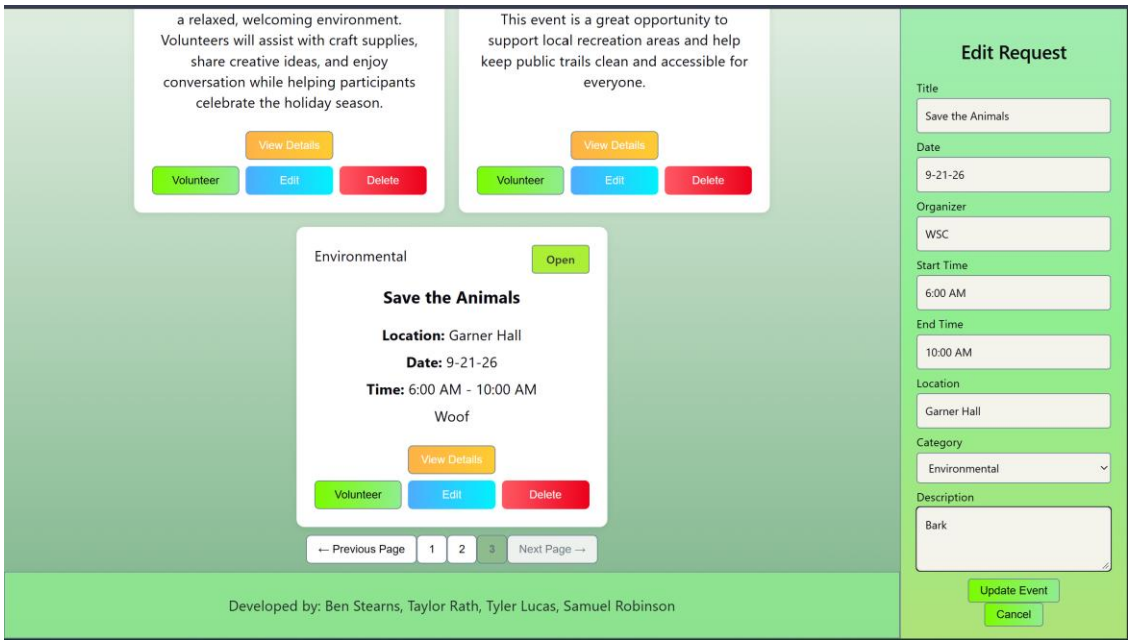
- Add a valid record
  - **Steps taken**: Use the form with valid data to add a new record
  - **Expected Result**: the new event should appear as a card on the last page and the stat's count should show one more event in total than before
  - **Pass/Fail**: Failed
  - Reason: a new id for the new event wasn't being created since the form doesn't provide this
  - **Action taken:** Updated the custom createEvent hook to create the next id number in sequence from the current list of events
  - **Result of correction**: passed
  - Screenshot:
    - 

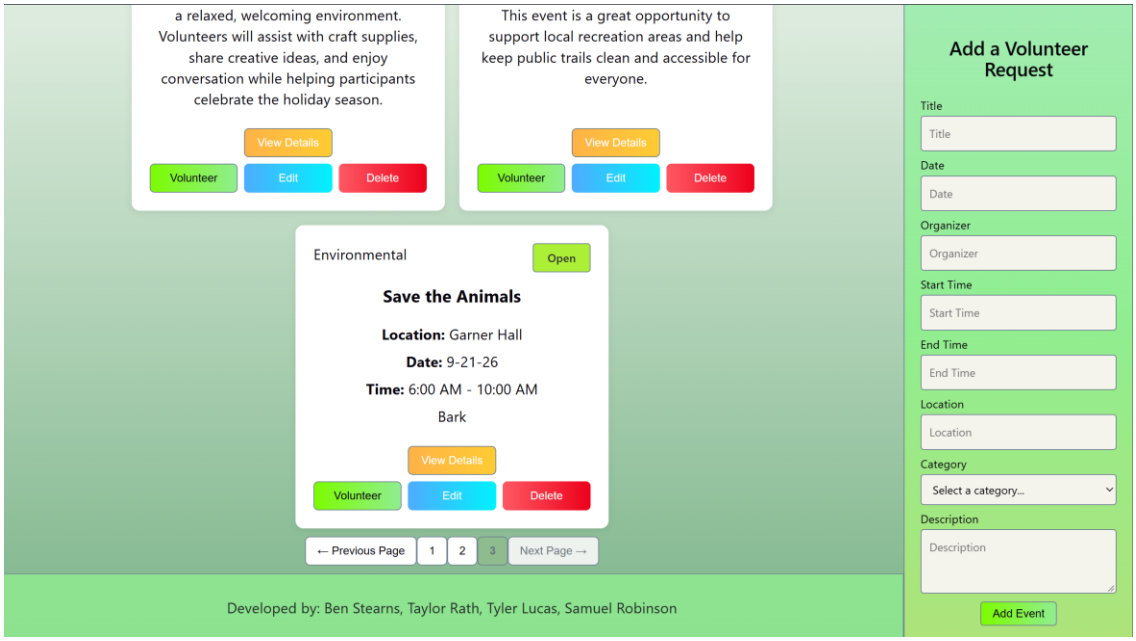
- 

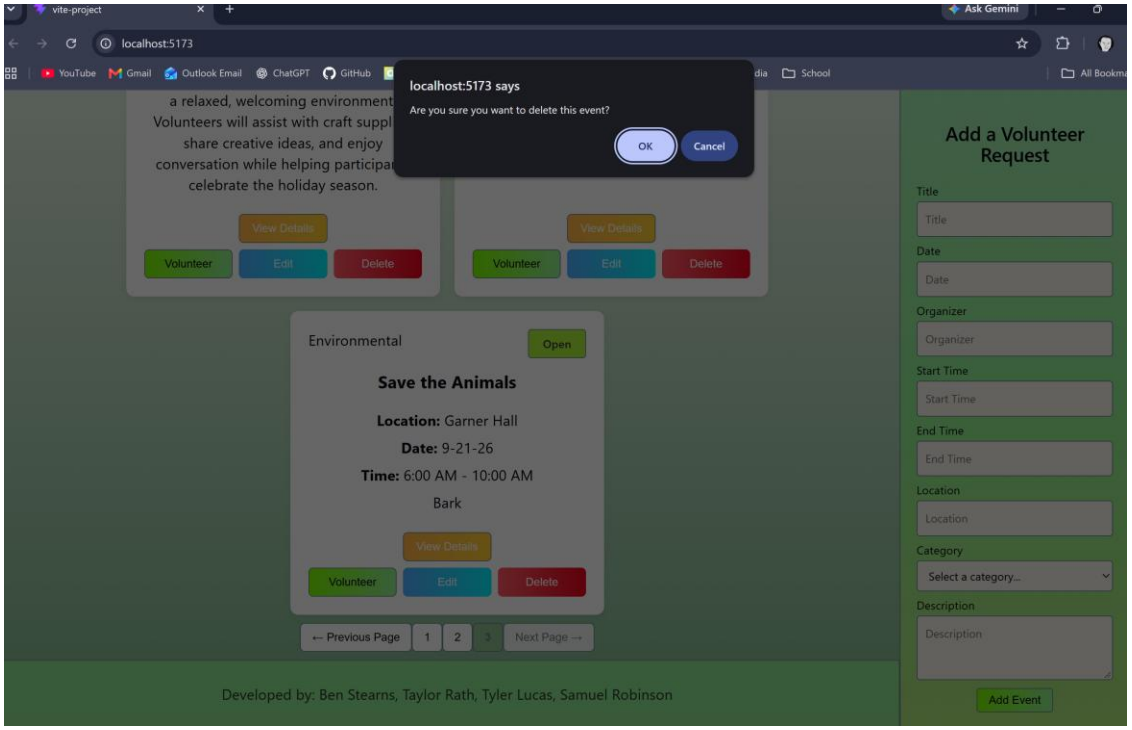
- Submit an empty required field
  - **Steps**: tested all fields one at a time to make sure they were required
  - **Results**: passed
  - **Screenshot**:

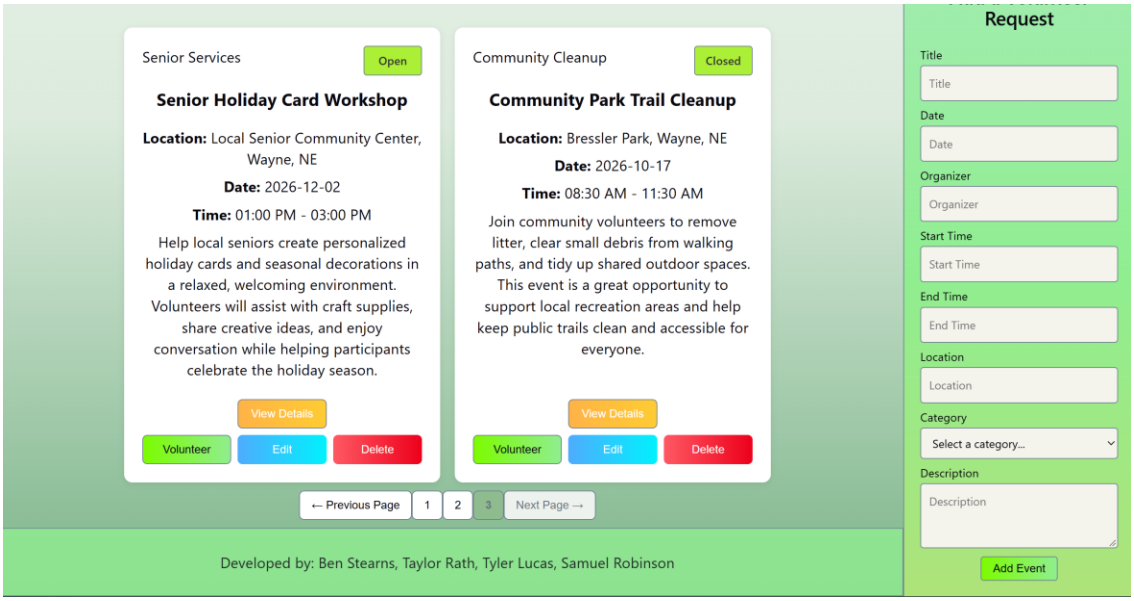


- Edit an existing record
  - **Steps**: Click the edit button and edited the description of an existing record, and clicked Update Event
  - **Results**: passed
  - **Screenshots**:

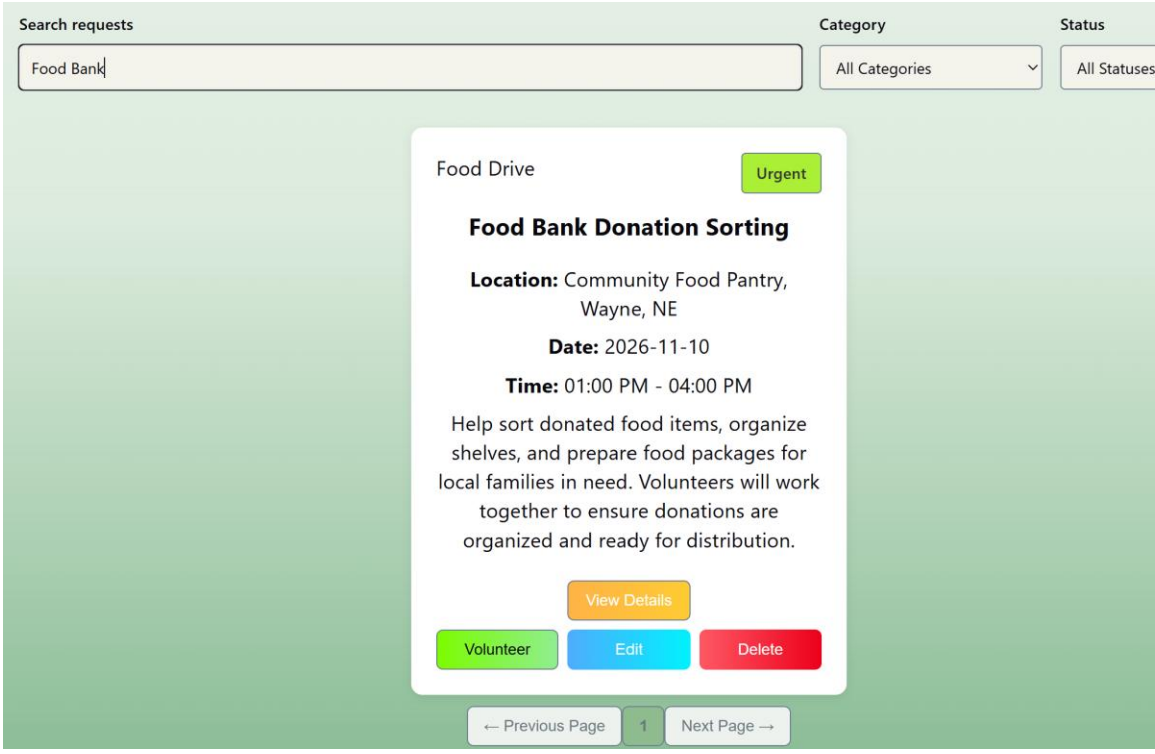


-


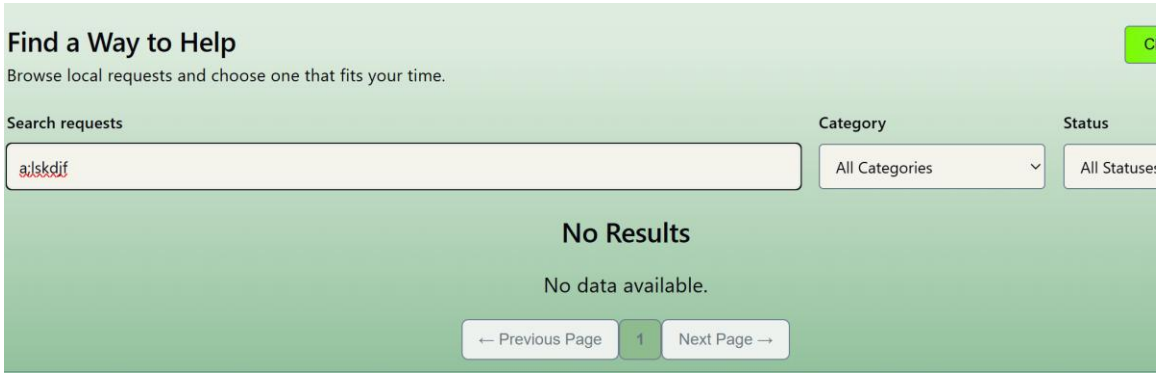
- Delete a record
  - **Steps**: click the delete button on one of the event cards
  - **Results**: passed
  - **Screenshots**:
    -


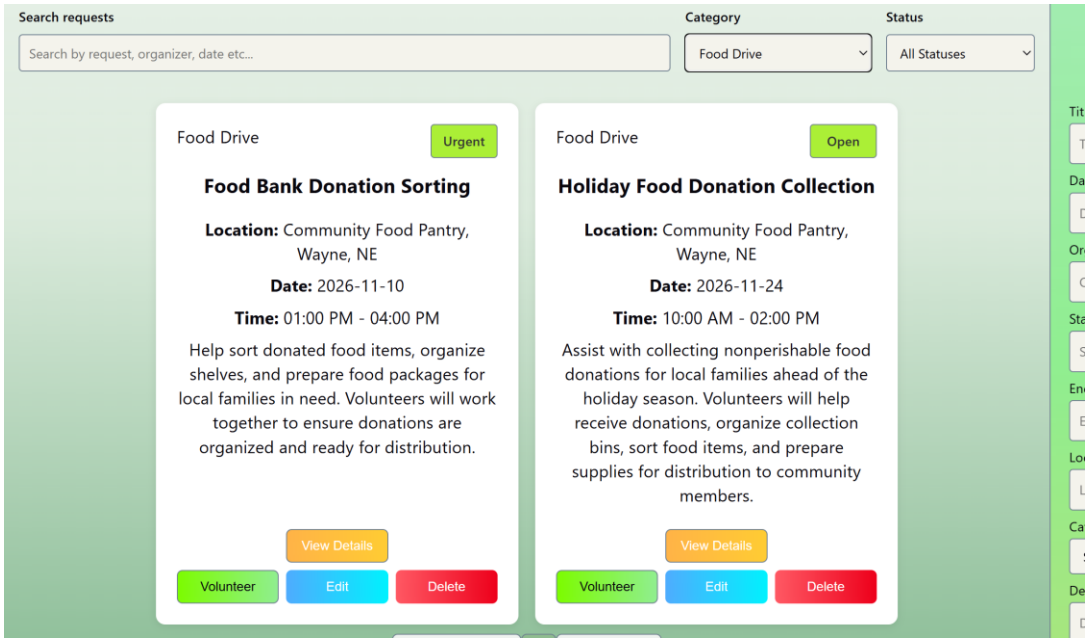


- Search for a known record
  - **Steps**: type in a known title for an event
  - **Results**: pass
  - **Screenshots**

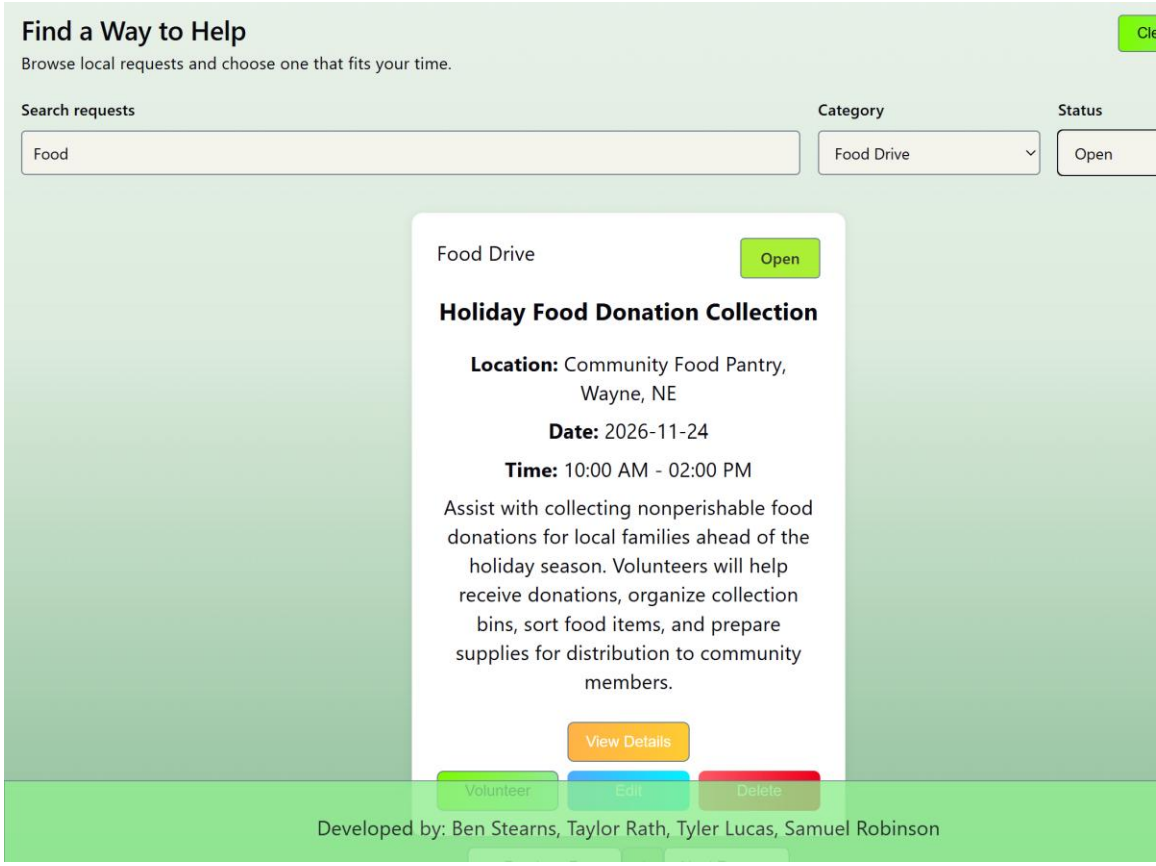


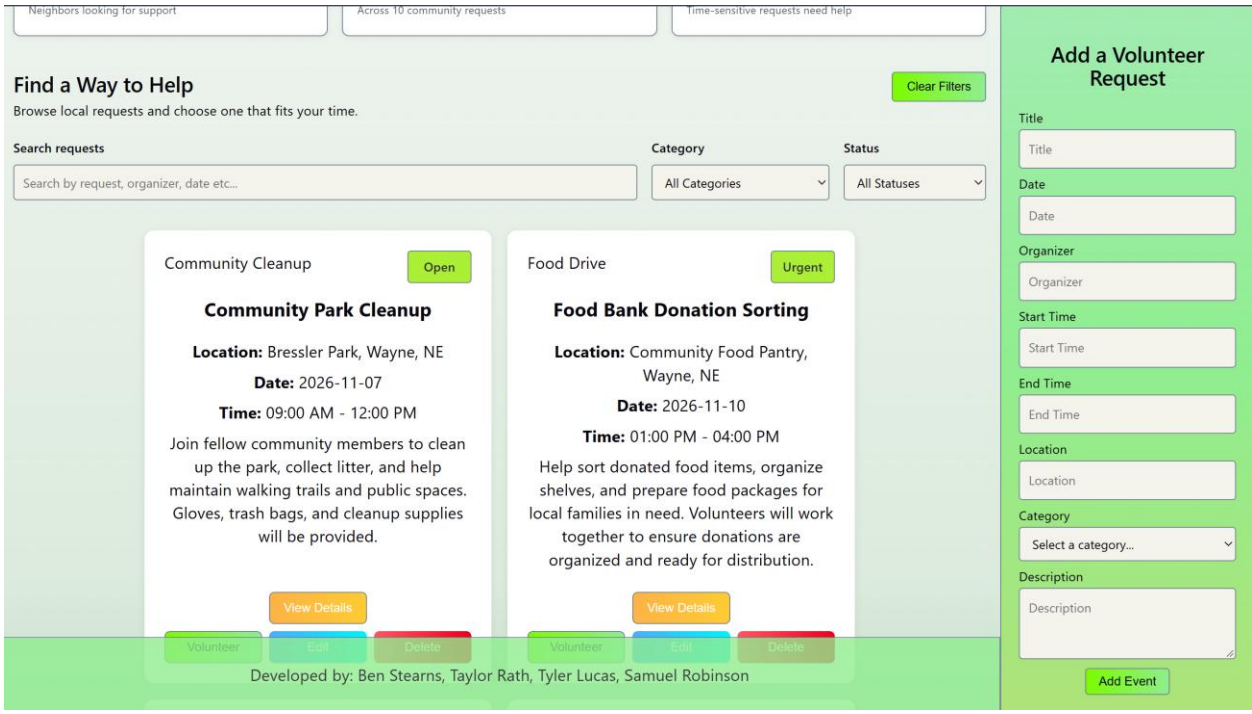
- Search for a non-existent record
  - **Steps**: type a search in the search input that shouldn't match anything
  - **Expected Result**: the No Results empty state component should render

    - **Pass/Fail**: pass
    - **Screenshots**
        - 

- Select a filter
    - **Steps**: select a category from the Category drop-down
    - **Expected Result**: Only events matching that category should display
    - **Pass/Fail**: pass
    - **Screenshots**
        - 

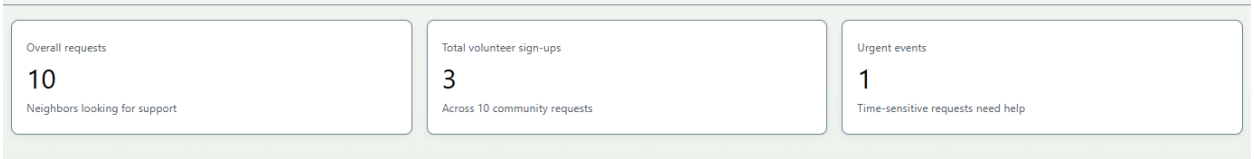
- Reset search and filters
    - **Steps**: type in a search query, select a category filter, and select a status filter and take note of the results. Then, click the Clear Filters button
    - **Expected result**: The search text clears, filters return to All Categories and All Statuses, and event list returns to displaying all events.
    - **Pass/Fail**: Pass
    - **Screenshots**





- **Event Stats Testing**

- **Calculates each event from overall Events, total events that volunteers signed up for, and urgent events**



- **Other Issues Encountered:**
  - An event's status wasn't being set correctly upon form submission since no field exists for that information in the form. This was correctly by updating the createEvent to set the status to Open by default if none was provided by the form.
  - Pagination took several attempts to get to work. Buttons we're working due forgetting to pass their functionality from the App component to the Pagination component. CSS selectors for disabling/enabling appropriate buttons didn't work at first due to not using the right class selectors. My algorithms weren't always accurate to determine the pages. Just looking at error logs + trial and error got us there though.

Part 6 — Deliverables

Submit one team package containing:

1. Project proposal: Application theme, purpose, target audience, and planned features.
2. Design document: Component diagram, interface sketch, state-management plan, and CRUD pseudocode.
3. React/Vite source code: Complete project folder excluding node_modules.
4. README.md: Project description, team members, installation instructions, and feature overview.
5. Testing document: Completed test cases and debugging notes.
6. Team contribution report: What each member developed and tested.

Team Class Demonstration

Each team will give a 7–10 minute demonstration that includes an overview of the application, working CRUD operations, search and filtering, component communication, useState, and useEffect.

I may ask individual students to explain or modify a small portion of their code.

Grading Rubric

100 points total

Category	Points
Proposal, design, and pseudocode	15
Component architecture and props	15
React state and event handling	15
Complete CRUD functionality	20
JSON, useEffect, search, and filtering	15
Interface quality and usability	5
Testing and documentation	10
Team demonstration and participation	5
Total	100

Individual Accountability

Although the application is a team submission, individual grades may be adjusted based on demonstrated understanding, documented contributions, and participation.

AI Usage Policy

This is a skills-review lab intended to demonstrate each student's understanding of the material.

You must write your own application code using class lectures, notes, documentation, and previous exercises.

Generative AI may not be used to generate components, CRUD implementations, or complete application solutions. Teams must maintain a brief development log describing their design decisions and problems encountered.

During the final demonstration, I may request a small live code change, such as adding a filter condition, modifying a component prop, or changing a state update.

The purpose of this lab is not to create the most elaborate application. It is to determine whether you can transfer what they learned while developing Campus Pulse into an entirely different application without being given the component structure or complete code.